POSTS AND TELECOMMUNICATIONS
INSTITUE OF TECHNOLOGY
INFORMATION TECHNOLOGY

INTERNSHIP

REPORT

WoxionChat

Name: NGUYEN PHAM TRUNG HIEU
Student ID: B23DCCE031
Class ID: B23CQCE04-B
Instructor: KIM NGOC BACH

Hanoi

Table of contents

1 Overview **2**

2 Technical Accomplishments **2**
2.1 Database Schema Design (NoSQL Integration) 2
2.2 Permission Logic (Role-Based Access Control) 2

3 Technology Stack **2**

4 Progress Evaluation **3**

5 Upcoming Milestone (Next Week) **3**

Table of contents

1 Overview

The primary objective for this week was to establish a robust, secure, and scalable User Management system using the **Django Framework**. The architecture focuses on a NoSQL integration to handle flexible user profiles and session tracking, ensuring a clear distinction between administrative roles (Boss/Superuser) and operational roles (Staff/Employees).

2 Technical Accomplishments

2.1 Database Schema Design (NoSQL Integration)

We implemented a document-based schema compatible with Django's authentication system.

- **User Entity:** An extension of Django's `AbstractUser` was created. Key fields include:
  - **Security:** Passwords are hashed using `pbkdf2_sha256` with 1,000,000 iterations for maximum resistance against brute-force attacks.
  - **Attributes:** Fields such as `username`, `email`, `first_name`, `last_name`, `is_active`, and `is_verified`.
  - **Roles:** A dedicated `role` field and a granular `permissions` list for employee-specific tasking.
- **User Session Entity:** A custom session tracking mechanism was developed to monitor active connections:
  - `session_key`: Unique identifier generated by the Django Session Engine.
  - `last_activity`: Real-time timestamp tracking of user interactions.
  - **Environment Tracking:** Logging of `ip_address` and `user_agent` (e.g., Chrome on Linux) to identify devices and prevent session hijacking.

2.2 Permission Logic (Role-Based Access Control)

A hierarchical system was established to differentiate access levels:

- **Administrative Level (The Boss):** Defined by `is_superuser` and `is_staff` flags. These users have full visibility into all active sessions, user agents, and system-wide configurations.
- **Operational Level (Staff):** Defined by `is_staff` but restricted by the `permissions` array. They have access to specific chat management tools without superuser privileges.

3 Technology Stack

- **Backend:** Django (Python-based framework).
- **Database:** MongoDB (utilizing BSON for flexible document storage).
- **Auth Mechanism:** Django Authentication Backend with PBKDF2 SHA256 hashing.
- **Session Engine:** Custom Django Middleware for IP and User-Agent logging.

4 Progress Evaluation

- **Status:** On Track.
- **Observations:** The integration of `last_activity` tracking allows for precise monitoring of system usage. The security flags (`is_verified`, `is_active`) provide the necessary levers for managing corporate account lifecycles.

5 Upcoming Milestone (Next Week)

1. **Admin Dashboard Customization:** Building a visual monitoring tool for "The Boss" to view live sessions.
2. **AI System Workflow Design:** Designing the end-to-end operational flow for the AI integration, including prompt orchestration, context management, and response filtering.
3. **Finetuning Data Pipeline:** Developing a specialized strategy for generating and structuring high-quality datasets to fine-tune the Large Language Model (LLM) for WoxionChat's specific domain.